

SECTION 7

CPU32 INSTRUCTIONS

This section describes the instructions provided for the CPU32. The CPU32 can execute object code from an MC68000 and MC68010 and many of the instructions of the MC68020.

There are three new instructions provided for the CPU32: enter background mode (BGND), low-power stop (LPSTOP), and table lookup and interpolate (TBLS, TBLSN, TBLU, and TBLUN). Table 7-1 lists the MC68020 instructions not supported by the CPU32.

Table 7-1. MC68020 Instructions Not Supported

Mnemonic	Description
BFCHG	Test Bit Field and Change
BFCLR	Test Bit Field and Clear
BFEXTS	Signed Bit Field Extract
BFEXTU	Unsigned Bit Field Extract
BFFFO	Bit Field Find First One
BFINS	Bit Field Insert
BFSET	Test Bit Field and Set
BFTST	Test Bit Field
CALLM	CALL Module
CAS	Compare and Swap Operands
CAS2	Compare and Swap Dual Operands
cpBcc	Branch on Coprocessor Condition
cpDBcc	Test Coprocessor Condition Decrement and Branch
cpGEN	Coprocessor General Function
cpRESTORE	Coprocessor Restore Function
cpSAVE	Coprocessor Save Function
cpScc	Set on Coprocessor Condition
cpTRAPcc	Trap on Coprocessor Condition
RTM	Return from Module
PACK	Pack BCD
UNPK	Unpack BCD

Addressing in the CPU32 is register oriented. Most instructions allow the results of the specified operation to be placed either in a register or directly in memory. This flexibility eliminates the need for extra instructions to store register contents in memory. Table 7- 2 lists the M68000 family addressing modes with cross-references to the MC68000, MC68010, CPU32, and MC68020. When referring to instructions in the previous sections, refer to Table 7-2 to identify the addressing modes available to the CPU32. Table 7-3 lists the instructions for the CPU32.

Table 7-2. M68000 Family Addressing Modes

Addressing Mode	Syntax	MC68000 MC68010	CPU32	MC68020
Register Indirect	Rn	X	X	X
Address Register Indirect	(An)	X	X	X
Address Register Indirect with Postincrement	(An) +	X	X	X
Address Register Indirect with Postdecrement	– (An)	X	X	X
Address Register Indirect with Displacement	(d ₁₆ ,An)	X	X	X
Address Register Indirect with Index (8-Bit Displacement)	(d ₈ ,An,Xn)	X	X	X
Address Register Indirect with Index (Base Displacement)	(d ₈ ,An,Xn*SCALE)		X	X
Memory Indirect with Postincrement	([bd,An],Xn, od)			X
Memory Indirect with Preincrement	([bd,An],Xn, od)			X
Absolute Short	(xxx).W	X	X	X
Absolute Long	(xxx).L	X	X	X
Program Counter Indirect with Displacement	(d ₁₆ ,PC)	X	X	X
Program Counter Indirect with Index (8-Bit Displacement)	(d ₈ ,PC,Xn)	X	X	X
Program Counter Indirect with Index (Base Displacement)	(d ₈ ,PC,Xn*SC ALE)		X	X
Immediate	# < data >	X	X	X
PC Memory Indirect with Postincrement	([bd,PC],Xn, od)			X
PC Memory Indirect with Predecrement	([bd,PC],Xn, od)			X

NOTE: Xn,SIZE*SCALE—Denotes index register n (data or address), the index size (W for word, L for long word and scale factor (1, 2, 4, or 8 for no-word, long-word, or 8 for quad- word scaling, respectively).
X—Supported

Table 7-3. CPU32 Instruction Set

Mnemonic	Description	Mnemonic	Description
ABCD	Add Decimal with Extend	MOVE	Move
ADD	Add	MOVEA	Move Address
ADDA	Add Address	MOVE from CCR	Move Condition Code Register
ADDI	Add Immediate	Move from SR	Move from Status Register
ADDQ	Add Quick	MOVE to SR	Move to Status Register
ADDX	Add with Extend	MOVE USP	Move User Stack Pointer
AND	Logical AND	MOVEC	Move Control Register
ANDI	Logical AND Immediate	MOVEC	Move Multiple Registers
ANDI to CCR	AND Immediate to Condition Code Register	MOVEM	Move Peripheral
ANDI to SR	AND Immediate to Status Register	MOVEP	Move Quick
ASL, ASR	Arithmetic Shift Left and Right	MOVEQ	Move Alternate Address Space
Bcc	Branch Conditionally	MULS	Signed Multiply
BCHG	Test Bit and Change	MULU	Unsigned Multiply
BCLR	Test Bit and Clear	NBCD	Negate Decimal with Extend
BGND	Enter Background Mode	NEG	Negate
BKPT	Breakpoint	NEGX	Negate with Extend
BRA	Branch	NOP	No Operation
BSET	Test Bit and Set	NOT	Logical Complement
BSR	Branch to Subroutine	PEA	Push Effective Address
BTST	Test Bit	RESET	Reset External Devices
CHK	Check Register Against Bound	ROL, ROR	Rotate Left and Right
CHK2	Check Register Against Upper and Lower Bound	ROXL, ROXR	Rotate with Extend Left and Right
CLR	Clear	RTD	Return and Deallocate
CMP	Compare	RTE	Return from Exception
CPMA	Compare Address	RTR	Return and Restore Codes
CMPI	Compare Immediate	RTS	Return from Subroutine
CMPM	Compare Memory to Memory	SBCD	Subtract Decimal with Extend
CMP2	Compare Register Against Upper and Lower Bounds	Scc	Set Conditionally
DBcc	Test Condition, Decrement, and Branch	STOP	Stop
DIVS, DIVSL	Signed Divide	SUB	Subtract
DIVU, DIVUL	Unsigned Divide	SUBA	Subtract Address
EOR	Logical Exclusive-OR	SUBI	Subtract Immediate
EORI	Logical Exclusive-OR Immediate	SUBQ	Subtract Quick
EORI to CCR	Exclusive-OR Immediate to Condition Code Register	SUBX	Subtract with Extend
EORI to SR	Exclusive-OR Immediate to Status Register	SWAP	Swap Register Words
EXG	Exchange Registers	TAS	Test Operand and Set
EXT, LSR	Sign-Extend	TBLS, TBSN	Signed/Unsigned Table Lookup and Interpolate
ILLEGAL	Take Illegal Instruction Trap	TBLU, TBLUN	Signed/Unsigned Table Lookup and Interpolate
JMP	Jump	TRAP	Trap
JSR	Jump to Subroutine	TRAPcc	Trap Conditionally
LEA	Load Effective Address	TRAPV	Trap an Overflow
LINK	Link and Allocate	TST	Test Operand
LPSTOP	Low Power Stop	UNLK	Unlink
LSL, LSR	Logical Shift Left and Right		

BGND

Enter Background Mode (CPU32)

BGND

Operation: If Background Mode Enabled
Then Enter Background Mode
Else Format/Vector Offset $\rightarrow$ – (SSP);
PC $\rightarrow$ – (SSP)
SR $\rightarrow$ – (SSP)
(Vector) $\rightarrow$ PC

Assembler

Syntax: BGND

Attributes: Size = (Unsize)

Description: The processor suspends instruction execution and enters background mode if background mode is enabled. The freeze output is asserted to acknowledge entrance into background mode. Upon exiting background mode, instruction execution continues with the instruction pointed to by the current program counter. If background mode is not enabled, the processor initiates illegal instruction exception processing. The vector number is generated to reference the illegal instruction exception vector. Refer to the appropriate user's manual for detailed information on background mode.

Condition Codes:

X	N	Z	V	C
—	—	—	—	—

X — Not affected.

N — Not affected.

Z — Not affected.

V — Not affected.

C — Not affected.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	0	1	0	1	1	1	1	1	0	1	0

LPSTOP

Low-Power Stop
(CPU32)

LPSTOP

Operation: If Supervisor State
Immediate Data → SR
Interrupt Mask → External Bus Interface (EBI)
STOP
Else TRAP

Assembler Syntax: LPSTOP # < data >

Attributes: Size = (Word) Privileged

Description: The immediate operand moves into the entire status register, the program counter advances to point to the next instruction, and the processor stops fetching and executing instructions. A CPU LPSTOP broadcast cycle is executed to CPU space \$3 to copy the updated interrupt mask to the external bus interface (EBI). The internal clocks are stopped.

Instruction execution resumes when a trace, interrupt, or reset exception occurs. A trace exception will occur if the trace state is on when the LPSTOP instruction is executed. If an interrupt request is asserted with a higher priority than the current priority level set by the new status register value, an interrupt exception occurs; otherwise, the interrupt request is ignored. If the bit of the immediate data corresponding to the S-bit is off, execution of the instruction will cause a privilege violation. An external reset always initiates reset exception processing.

Condition Codes:
Set according to the immediate operand.

Instruction Format:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
IMMEDIATE DATA															

Instruction Fields:
Immediate field—Specifies the data to be loaded into the status register.

TBLS TBLSN

Table Lookup and Interpolate (Signed) (CPU32)

TBLS TBLSN

Operation: Rounded:
$$\text{ENTRY}(n) + \{(\text{ENTRY}(n + 1) - \text{ENTRY}(n)) \times \text{Dx } 7 - 0\} \div 256 \rightarrow \text{Dx}$$

Unrounded:
$$\text{ENTRY}(n) \times 256 + \{(\text{ENTRY}(n + 1) - \text{ENTRY}(n)) \times \text{Dx } 7 - 0\} \rightarrow \text{Dx}$$

Where ENTRY(n) and ENTRY(n + 1) are either:

1. Consecutive entries in the table pointed to by the < ea > and indexed by Dx 15 – 8 π SIZE or;
2. The registers Dym, Dyn respectively.

Assembler	TBLS. < size > < ea > ,Dx	Result rounded
Syntax:	TBLSN. < size > < ea > ,Dx	Result not rounded
	TBLS. < size > Dym:Dyn, Dx	Result rounded
	TBLSN. < size > Dym:Dyn, Dx	Result not rounded

Attributes: Size = (Byte, Word, Long)

Description: The TBLS and TBLSN instructions allow the efficient use of piecewise linear compressed data tables to model complex functions. The TBLS instruction has two modes of operation: table lookup and interpolate mode and data register interpolate mode.

For table lookup and interpolate mode, data register Dx 15 – 0 contains the independent variable X. The effective address points to the start of a signed byte, word, or long-word table containing a linearized representation of the dependent variable, Y, as a function of X. In general, the independent variable, located in the low-order word of Dx, consists of an 8-bit integer part and an 8-bit fractional part. An assumed radix point is located between bits 7 and 8. The integer part, Dx 15 – 8, is scaled by the operand size and is used as an offset into the table. The selected entry in the table is subtracted from the next consecutive entry. A fractional portion of this difference is taken by multiplying by the interpolation fraction, Dx 7 – 0. The adjusted difference is then added to the selected table entry. The result is returned in the destination data register, Dx.

TBLS TBLSN

Table Lookup and Interpolate (Signed) (CPU32)

TBLS TBLSN

For register interpolate mode, the interpolation occurs using the Dym and Dyn registers in place of the two table entries. For this mode, only the fractional portion, Dx 7 – 0, is used in the interpolation, and the integer portion, Dx 15 – 8, is ignored. The register interpolation mode may be used with several table lookup and interpolations to model multidimensional functions.

Signed table entries range from -2^{n-1} to $2^{n-1} - 1$; whereas, unsigned table entries range from 0 to $2^n - 1$ where n is 8, 16, or 32 for byte, word, and long-word tables, respectively.

Rounding of the result is optionally selected via the "R" instruction field. If R = 0 (TABLE), the fractional portion is rounded according to the round-to-nearest algorithm. The following table summarizes the rounding procedure:

Adjusted Difference Fraction	Rounding Adjustment
$\leq -1/2$	- 1
$> -1/2$ and $< 1/2$	+ 0
$\geq 1/2$	+ 1

The adjusted difference is then added to the selected table entry. The rounded result is returned in the destination data register, Dx. Only the portion of the register corresponding to the selected size is affected.

	31	24	23	16	15	8	7	0
BYTE	UNAFFECTED	UNAFFECTED	UNAFFECTED	UNAFFECTED	UNAFFECTED	UNAFFECTED	RESULT	RESULT
WORD	UNAFFECTED	UNAFFECTED	UNAFFECTED	UNAFFECTED	RESULT	RESULT	RESULT	RESULT
LONG	RESULT	RESULT	RESULT	RESULT	RESULT	RESULT	RESULT	RESULT



TBLS
TBLSN

Table Lookup and Interpolate (Signed)
(CPU32)

TBLS
TBLSN

If R = 1 (TABLENR), the result is returned in register Dx without rounding. If the size is byte, the integer portion of the result is returned in Dx 15 – 8; the integer portion of a word result is stored in Dx 23 – 8; the least significant 24 bits of a long result are stored in Dx 31 – 8. Byte and word results are sign-extended to fill the entire 32-bit register.

	31	24	23	16	15	8	7	0
BYTE	SIGN-EXTENDED							
WORD	SIGN-EXTENDED							
LONG	RESULT							

NOTE

The long-word result contains only the least significant 24 bits of integer precision.

For all sizes, the 8-bit fractional portion of the result is returned to the low byte of the data register, Dx 7 – 0. User software can make use of the fractional data to reduce cumulative errors in lengthy calculations or implement rounding algorithms different from that provided by other forms of TBLS. The previously described assumed radix point places two restrictions on the programmer:

1. Tables are limited to 257 entries in length.
2. Interpolation resolution is limited to 1/256, the distance between consecutive table entries. The assumed radix point should not, however, be construed by the programmer as a requirement that the independent variable be calculated as a fractional number in the range $0 < \pi < 255$. On the contrary, X should be considered an integer in the range $0 < \pi < 65535$, realizing that the table is actually a compressed representation of a linearized function in which only every 256th value is actually stored in memory.

TBLS TBLSN

Table Lookup and Interpolate (Signed) (CPU32)

TBLS TBLSN

Condition Codes:

X	N	Z	V	C
—	*	*	*	0

- X — Not affected.
- N — Set if the most significant bit of the result is set; cleared otherwise.
- Z — Set if the result is zero; cleared otherwise.
- V — Set if the integer portion of an unrounded long result is not in the range, $-(2^{23}) \leq \text{Result} \leq (2^{23}) - 1$; cleared otherwise.
- C — Always cleared.

Instruction Format:

TABLE LOOKUP AND INTERPOLATE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	REGISTER Dx			1	R	0	1	SIZE		0	0	0	0	0	0

DATA REGISTER INTERPOLATE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	0	0	REGISTER Dym		
0	REGISTER Dx			1	R	0	1	SIZE		0	0	0	REGISTER Dyn		

TBLS TBLSN

Table Lookup and Interpolate (Signed) (CPU32)

TBLS TBLSN

Instruction Fields:

Effective address field (table lookup and interpolate mode only)—Specifies the destination location. Only control alterable addressing modes are allowed as listed in the following table:

Addressing Mode	Mode	Register	Addressing Mode	Mode	Register
Dn	—	—	(xxx).W	111	000
An	—	—	(xxx).L	111	001
(An)	—	—	# < data >	—	—
(An) +	—	—			
– (An)	100	reg. number:An			
(d ₁₆ ,An)	101	reg. number:An	(d ₁₆ ,PC)	111	010
(d ₈ ,An,Xn)	110	reg. number:An	(d ₈ ,PC,Xn)	111	011
(bd,An,Xn)	110	reg. number:An	(bd,PC,Xn)	111	011

Size Field—Specifies the size of operation.

- 00 — Byte Operation
- 01 — Word Operation
- 10 — Long Operation

Register field—Specifies the destination data register, Dx. On entry, the register contains the interpolation fraction and entry number.

Dym, Dyn field—If the effective address mode field is nonzero, this operand register is unused and should be zero. If the effective address mode field is zero, the surface interpolation variant of this instruction is implied, and Dyn specifies one of the two source operands.

Rounding mode field—The R-bit controls the rounding of the final result. When R = 0, the result is rounded according to the round-to-nearest algorithm. When R = 1, the result is returned unrounded.

TBLU TBLUN

Table Lookup and Interpolation (Unsigned) (CPU32)

TBLU TBLUN

Operation:

Rounded:

$$\text{ENTRY}(n) + \{(\text{ENTRY}(n + 1) - \text{ENTRY}(n)) \times \text{Dx } 7 - 0\} \div 256 \rightarrow \text{Dx}$$

Unrounded:

$$\text{ENTRY}(n) \times 256 + \{(\text{ENTRY}(n + 1) - \text{ENTRY}(n)) \times \text{Dx } 7 - 0\} \rightarrow \text{Dx}$$

Where ENTRY(n) and ENTRY(n + 1) are either:

1. Consecutive entries in the table pointed to by the < ea > and indexed by Dx 15 – 8 π SIZE or;
2. The registers Dym, Dyn respectively

Assembler

TBLU. < size > < ea > ,Dx

Result rounded

Syntax:

TBLUN. < size > < ea > ,Dx

Result not rounded

TBLU. < size > Dym:Dyn, Dx

Result rounded

TBLUN. < size > Dym:Dyn, Dx

Result not rounded

Attributes:

Size = (Byte, Word, Long)

Description: The TBLU and TBLUN instructions allow the efficient use of piecewise linear, compressed data tables to model complex functions. The TBLU instruction has two modes of operation: table lookup and interpolate mode and data register interpolate mode.

For table lookup and interpolate mode, data register Dx 15 – 0 contains the independent variable X. The effective address points to the start of a unsigned byte, word, or long-word table containing a linearized representation of the dependent variable, Y, as a function of X. In general, the independent variable, located in the low-order word of Dx, consists of an 8-bit integer part and an 8-bit fractional part. An assumed radix point is located between bits 7 and 8. The integer part, Dx 15 – 8, is scaled by the operand size and is used as an offset into the table. The selected entry in the table is subtracted from the next consecutive entry. A fractional portion of this difference is taken by multiplying by the interpolation fraction, Dx 7 – 0. The adjusted difference is then added to the selected table entry. The result is returned in the destination data register, Dx.

TBLU

TBLUN

Table Lookup and Interpolation (Unsigned) (CPU32)

TBLU

TBLUN

For register interpolate mode, the interpolation occurs using the Dym and Dyn registers in place of the two table entries. For this mode, only the fractional portion, Dx 7 – 0, is used in the interpolation and the integer portion, Dx 15 – 8, is ignored. The register interpolation mode may be used with several table lookup and interpolations to model multidimensional functions.

Signed table entries range from -2^{n-1} to $2^{n-1} - 1$; whereas, unsigned table entries range from 0 to $2^n - 1$ where n is 8, 16, or 32 for byte, word, and long-word tables, respectively. The unsigned and unrounded table results will be zero-extended instead of sign-extended.

Rounding of the result is optionally selected via the "R" instruction field. If R = 0 (TABLE), the fractional portion is rounded according to the round-to-nearest algorithm. The rounding procedure can be summarized by the following table:

Adjusted Difference Fraction	Rounding Adjustment
$\geq 1/2$	+ 1
$< 1/2$	+ 0

The adjusted difference is then added to the selected table entry. The rounded result is returned in the destination data register, Dx. Only the portion of the register corresponding to the selected size is affected.

	31	24	23	16	15	8	7	0
BYTE	UNAFFECTED				UNAFFECTED			
WORD	UNAFFECTED				RESULT			
LONG	RESULT				RESULT			

If R = 1 (TBLUN), the result is returned in register Dx without rounding. If the size is byte, the integer portion of the result is returned in Dx 15 – 8; the integer portion of a word result is stored in Dx 23 – 8; the least significant 24 bits of a long result are stored in Dx 31 – 8. Byte and word results are sign-extended to fill the entire 32-bit register.

TBLU TBLUN

Table Lookup and Interpolation (Unsigned) (CPU32)

TBLU TBLUN

	31	24	23	16	15	8	7	0
BYTE	SIGN-EXTENDED		SIGN-EXTENDED		RESULT		FRACTION	
WORD	SIGN-EXTENDED		RESULT		RESULT		FRACTION	
LONG	RESULT		RESULT		RESULT		FRACTION	

NOTE

The long-word result contains only the least significant 24 bits of integer precision.

For all sizes, the 8-bit fractional portion of the result is returned in the low byte of the data register, Dx 7 – 0. User software can make use of the fractional data to reduce cumulative errors in lengthy calculations or implement rounding algorithms different from that provided by other forms of TBLU. The previously described assumed radix point places two restrictions on the programmer:

1. Tables are limited to 257 entries in length.
2. Interpolation resolution is limited to 1/256, the distance between consecutive table entries. The assumed radix point should not, however, be construed by the programmer as a requirement that the independent variable be calculated as a fractional number in the range $0 \leq X \leq 255$. On the contrary, X should be considered to be an integer in the range $0 \leq X \leq 65535$, realizing that the table is actually a compressed representation of a linearized function in which only every 256th value is actually stored in memory.

Condition Codes:

X	N	Z	V	C
—	*	*	*	0

- X — Not affected.
- N — Set if the most significant bit of the result is set; cleared otherwise.
- Z — Set if the result is zero; cleared otherwise.
- V — Set if the integer portion of an unrounded long result is not in the range, $-(2^{23}) \leq \text{Result} \leq (2^{23}) - 1$; cleared otherwise.
- C — Always cleared.



TBLU TBLUN Table Lookup and Interpolation (Unsigned) (CPU32)

TBLU
TBLUN

Instruction Format:

TABLE LOOKUP AND INTERPOLATE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	EFFECTIVE ADDRESS					
										MODE			REGISTER		
0	REGISTER Dx			0	R	0	1	SIZE	0	0	0	0	0	0	0

DATA REGISTER INTERPOLATE

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	1	0	0	0	0	0	0	0	0	REGISTER Dym		
0	REGISTER Dx			0	R	0	0	SIZE		0	0	0	REGISTER Dyn		

Instruction Fields:

Effective address field (table lookup and interpolate mode only)—Specifies the destination location. Only control alterable addressing modes are allowed as listed in the following table:

Addressing Mode	Mode	Register
Dn	—	—
An	—	—
(An)	010	reg. number:An
(An) +	—	—
– (An)	100	reg. number:An
(d ₁₆ ,An)	101	reg. number:An
(d ₈ ,An,Xn)	110	reg. number:An
(bd,An,Xn)	110	reg. number:An

Addressing Mode	Mode	Register
(xxx).W	111	000
(xxx).L	111	001
# < data >	—	—
(d ₁₆ ,PC)	111	010
(d ₈ ,PC,Xn)	111	011
(bd,PC,Xn)	111	011

Size field—Specifies the size of operation.

- 00 — Byte Operation
- 01 — Word Operation
- 10 — Long Operation

TBLU

TBLUN

Table Lookup and Interpolation (Unsigned) (CPU32)

TBLU

TBLUN

Register field—Specifies the destination data register, Dx. On entry, the register contains the interpolation fraction and entry number.

Dym, Dyn field—If the effective address mode field is nonzero, this operand register is unused and should be zero. If the effective address mode field is zero, the surface interpolation variant of this instruction is implied, and Dyn specifies one of the two source operands.

Rounding mode field—The R-bit controls the rounding of the final result. When R = 0, the result is rounded according to the round-to-nearest algorithm. When R = 1, the result is returned unrounded.

